Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Tutorial

Week 4

Question 1

Suppose we are given a 2D-array A of size m rows by n columns. An element in the array $A[i][j]$ is called a "peak" if it is **greater than or equal to** its adjacent neighbours (if they exist -- an element is always considered greater than or equal to non-existent elements). For example, the 8 in the middle is a peak in the following array.

*	*	5	*
*	8	8	3
*	*	2	*

Consider the following algorithm to return any "peak":

```
Find2DPeak(A):

    If A only has a column, return the maximal element of the column

    Otherwise:

        Select the middle column of the A

        Find the maximal element of the column

        If the maximal element is a peak, return that element

        Else

            p1 = Find2DPeak(right half of A excluding middle col)

            p2 = Find2DPeak(left half of A excluding middle col)

            If p1 or p2 is a peak, return either one, otherwise return None
```

What is the runtime of the algorithm?

- $\Theta(mn)$
- $\Theta(m \lg n)$
- $\Theta(\lg m \lg n)$
- $\Theta(m^2 \lg n)$

Question 1

Answer: A

Consider the *number of times a column is processed*.

- In the divide step, the middle column is processed, and no column is processed in the combined step. $O(1)$
- The number of columns is halved in the recursive step
- Two subproblems are solved.

Question 1

The recurrence for the number of times a column is processed is

$T(n) = 2T(n/2) + 1$

To solve using the Master theorem, we check

$f(n) = \Theta(1) = O(n^{\lg 2 + \varepsilon})$ giving

$T(n) = \Theta(n)$ by case 1.

And the time to process a column is $\Theta(m)$ so total time is $\Theta(mn)$.

Question 2

Suppose we are given a 2D-array A of size m rows by n columns. An element in the array $A[i][j]$ is called a "peak" if it is **greater than or equal to** its adjacent neighbours (if they exist -- an element is always considered greater than or equal to non-existent elements). For example, the 8 in the middle is a peak in the following array.

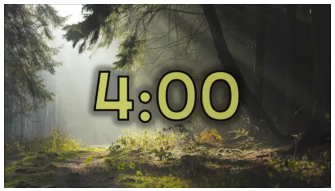
*	*	5	*
*	8	8	3
*	*	2	*

Consider the following algorithm to return any "peak":

```
Find2DPeak(A):

    If A only has a column, return the maximal element of the column

    Otherwise:

        Select the middle column of the A

        Find the maximal element of the column

        If the maximal element is a peak, return that element

        Else if the maximal element is smaller than the element on its right

            Recurse on right half of A (excluding middle column)

        Else recurse on left half of A (excluding middle column)
```

What is the runtime of the algorithm? Is the algorithm correct (always returns a peak)?

- $\Theta(mn)$ and correct.
- $\Theta(m \lg n)$ and correct.
- $\Theta(mn)$ but sometimes returns the wrong answer.
- $\Theta(m \lg n)$ but sometimes returns the wrong answer.



Question 2

Answer: B. First the runtime.

Consider the *number of times a column is processed*.

- In the divide step, the middle column is processed, and no column is processed in the combined step. $O(1)$
- The number of columns is halved in the recursive step
- **One** subproblem is solved.

Question 2

The recurrence for the number of times a column is processed is

$T(n) = T(n/2) + 1$

To solve using the Master theorem, we check

$f(n) = \Theta(1) = \Theta\ (n^{\lg 1} \log^0 n)$ giving

$T(n) = \Theta(\lg n)$ by case 2.

And the time to process a column is $\Theta(m)$ so total time is $\Theta(m \lg n)$.

Question 2

Now, we argue that the algorithm correctly returns a peak.

The base case of a single column is correct as the maximum element is selected and elements outside the array are considered smaller.

Question 2

Assume that the algorithm works correctly for arrays with $k < n$ columns.

If the maximum element in the middle column is a peak, the algorithm returns that element correctly.

Otherwise it recurses on the subproblem on one side. By the induction hypothesis it returns the correct answer when the subproblem is viewed as a complete 2D array.

Question 2

However, the subproblem is a subarray within the original array. If the returned element by the recursive call is not next to the middle column, then it is definitely a peak.

But the inductive hypothesis does not imply that it is a peak if it is next to the middle column, so we need to separately argue that it is a peak.

Question 2

Let the largest element in the middle column be $c_{i,j}$.

Assume $c_{k,j+1}$ is the element returned by the recursive call.

We know that $c_{k,j+1}$ is the largest element in column j+1. Then

$c_{k,j+1} \geq c_{i,j+1}$ ($c_{k,j+1}$ largest in the column)

$\geq c_{i,j}$ (larger neigh. of $c_{i,j}$ recursively picked)

$\geq c_{k,j}$ ($c_{i,j}$ largest in column)

So $c_{k,j+1}$ is a peak.

Question 3

Consider the following variant of merge sort where in the merge step:

A new sorted array is constructed by

1. Constructing a balanced tree out of the first array
2. Inserting the elements from the second array into the balanced tree one at a time
3. Doing an in-order walk of the balanced tree to construct a new sorted array

What is the recurrence and runtime of this variant of merge sort?

- $T(n) = 2T(n/2) + \Theta(n),\ T(n) = \Theta(n \log n)$
- $T(n) = 2T(n/2) + \Theta(1),\ T(n) = \Theta(n)$
- $T(n) = 2T(n/2) + \Theta(n^2),\ T(n) = \Theta(n^2)$
- $T(n) = 2T(n/2) + \Theta(n \lg n),\ T(n) = \Theta(n \lg^2 n)$

Question 3

Answer: D

The difference from the usual merge sort is that the merge routine use for the combine step now takes $\Theta(n \log n)$ time.

This gives the recurrence

$T(n) = 2T(n/2) + \Theta(n \log n)$

To solve using the Master theorem, we check $f(n) = \Theta(n^{\lg 2} \log n)$ giving $T(n) = \Theta(n \log^2 n)$ by case 2.